

Rapport detaille de realisation

Module NoSQL Security — Seance 4

NoSQL Injection, requetes sures, validation applicative et projection minimale

Objet du document. Ce rapport reconstruit le travail realise pendant la Seance 4 a partir du support de cours HTML, de l'archive de captures d'ecran et de la structure du rapport de reference de la Seance 2. L'objectif est de produire un livrable propre a l'impression, detaille, auditable et coherent avec les preuves terminal fournies.

Support pedagogique	seance4_nosql_injection_securite.html
Rapport modele	mongodb_lab2_report.pdf / mongodb_lab2_report.tex
Preuves visuelles	screenshots.zip (26 captures)
Stack applicatif	Node.js + Express + driver MongoDB
Base de travail	soc_lite, puis adaptation legere nosqli_lab.items
Environnement observe	Kali / WSL, mongosh, curl, python3 -m json.tool
Realisateur	Mohammed Zguioui

Table des matières

1	Resume executif	3
2	Sources exploitees et methode de reconstruction	3
2.1	Plan pedagogique reconstitue	3
3	Environnement et corpus manipule	4
3.1	Architecture applicative observee	4
3.2	Collections et documents utilises	4
4	Travail realise pendant la seance	4
4.1	Partie 1 — Initialisation du corpus et verification du comportement normal	4
4.2	Partie 2 — Demonstration controlee des injections	6
4.2.1	Refus du mauvais mot de passe	7
4.2.2	Injection <code>\$ne</code> sur le login	7
4.2.3	Injection <code>\$regex</code> sur la recherche de logs	7
4.2.4	Mass assignment et champs non autorises	8
5	Correction applicative : validation, <code>safeFilter</code> et conservation des usages legitimes	9
5.1	Validation de type	9
5.2	Construction explicite du filtre	9
5.3	Verification de non-regression fonctionnelle	10
6	Projection minimale et principe Least Data	11
6.1	Observation du probleme sans projection	11
6.2	Lecture avec projection minimale	12
7	Execution du devoir maison	13
7.1	Test 1 — Injection <code>\$gt</code> sur les logs	13
7.2	Test 2 — Champ non prevu dans le login	13
7.3	Test 3 — Fonction <code>hasNoMongoOperators()</code>	14
7.4	Test 4 — Effet de la projection avec et sans <code>project()</code>	14
7.5	Test 5 — Endpoint <code>/users/update-email</code>	15
8	Checklist securite remplie a partir des preuves	16
9	Analyse critique et enseignements	17
9.1	1. Le probleme est applicatif avant d’etre base de donnees	17
9.2	2. L’injection NoSQL n’est pas une injection SQL traduite	17

9.3	3. La projection est une defense independante	17
9.4	4. Mongoose ou un ORM/ODM ne remplacent pas la validation	17
10	Conclusion	17
A	Extraits representatifs des commandes executees	17
A.1	Requetes normales	17
A.2	Payloads d'injection	18
A.3	Projection minimale	18
A.4	Endpoint <code>update-email</code> securise	18

1 Resume executif

La Seance 4 a ete realisee autour d'un objectif principal : comprendre comment une API Node.js peut transformer une entree utilisateur en filtre MongoDB dangereux, puis mettre en place une construction de requetes sure. Les captures montrent une progression complete : peuplement du jeu de donnees, verification des requetes normales, reproduction controlee des injections `$ne`, `$regex` et `$gt`, correction par validation de type et filtrage des operateurs, application d'une projection minimale, puis execution des tests du devoir maison.

Le travail met en evidence une difference fondamentale avec les injections SQL classiques. Dans MongoDB, la charge utile malveillante ne cherche pas forcement a casser une chaine de caracteres : elle injecte directement des operateurs dans un objet JSON. Les exemples observes confirment ce point : un mot de passe remplace par `{"$ne":""}` contourne l'authentification sur l'API vulnérable, tandis qu'une recherche `{"src_ip":{"$regex":""}}` elargit la requete et retourne davantage de logs que la recherche exacte.

La correction mise en place repose sur quatre mecanismes complementaires : validation stricte des types, construction explicite d'un `safeFilter`, rejet des cles commençant par `$`, et projection minimale des champs retournes. Les tests finaux montrent que les requetes legitimes restent fonctionnelles, tandis que les payloads d'injection retournent des erreurs applicatives avant d'atteindre MongoDB.

Synthese en une phrase. Le TP demontre que le durcissement MongoDB ne suffit pas si l'API construit ses filtres avec `req.body` ; la defense efficace se situe dans le code applicatif, par validation, allow-list, filtrage des operateurs et minimisation des champs exposes.

2 Sources exploitees et methode de reconstruction

Le rapport reprend la structure du livrable de Seance 2 : resume executif, sources, environnement, deroule des TPs, execution du devoir, checklist, analyse critique et annexes de commandes. La reconstruction s'appuie sur trois sources principales :

- le support de Seance 4, qui definit le plan pedagogique en quatre parties : comprehension, demonstration, correction et projection minimale ;
- l'archive `screenshots.zip`, qui contient 26 preuves visuelles de commandes `mongosh`, requetes `curl`, sorties JSON et tests Node REPL ;
- le rapport de Seance 2, utilise comme modele de forme : page de garde, table des matieres, sections analytiques, figures commentees et annexes.

La methode suivie consiste a regrouper les captures par intention technique : initialisation des donnees, comportements normaux, injections observees, correction, projection, puis devoir maison. Les figures inserees dans ce rapport ne sont pas de simples illustrations : elles servent de preuves d'execution et sont commentees dans le texte.

2.1 Plan pedagogique reconstitue

Le support de Seance 4 structure le travail comme suit :

1. Partie 1 — comprendre la NoSQL Injection et le risque lie a `find(req.body)` ;
2. Partie 2 — reproduire des injections controlees : `$ne`, `$regex`, `$gt` et mass assignment ;
3. Partie 3 — corriger l'API par validation, allow-list et `safeFilter` ;

4. Partie 4 — appliquer une projection minimale et verifier l'absence de champs sensibles ;
5. Devoir maison — executer cinq tests complementaires et repondre a trois questions de reflexion.

3 Environnement et corpus manipule

3.1 Architecture applicative observee

L'environnement observe correspond a une API locale exposee sur le port 3000. Les appels sont effectues depuis un terminal Kali / WSL avec `curl`, puis les reponses JSON sont formatees au moyen de `python3 -m json.tool`. La base MongoDB est interrogee avec `mongosh`.

Deux niveaux apparaissent dans les preuves :

- le schema pedagogique original du support, avec `soc_lite`, `users` et `security_logs` ;
- une adaptation legere demandee pendant l'execution, avec une base `nosqli_lab` et une collection unique `items`, ou les documents sont distingues par le champ `kind`.

Cette adaptation conserve l'objectif pedagogique : il ne s'agit pas de complexifier la modelisation, mais d'isoler le comportement de securite de l'API.

3.2 Collections et documents utilises

Element	Role fonctionnel observe
<code>users</code>	Comptes de test pour l'authentification vulnerable et securisee : <code>alice</code> , <code>bob</code> , <code>carol</code> .
<code>security_logs</code>	Logs de securite utilises pour demontrer la recherche normale, l'exfiltration via <code>\$regex</code> et la projection.
<code>items</code>	Collection unique de l'adaptation legere ; contient les documents <code>kind:"user"</code> et <code>kind:"log"</code> .
<code>password_hash</code> <code>internal_note</code>	/ Champs sensibles injectes volontairement dans les logs pour verifier l'effet de la projection minimale.

4 Travail realise pendant la seance

4.1 Partie 1 — Initialisation du corpus et verification du comportement normal

La premiere etape a consiste a creer un petit referentiel d'utilisateurs. La capture montre l'insertion de trois comptes simplifiees : `alice`, `bob` et `carol`, avec leurs mots de passe de test et leurs roles. Une verification par projection confirme la presence des utilisateurs attendus.

```
soc_lite> db.users.insertMany([
|   { username: "alice", password: "motdepasse123", role: "analyst" },
|   { username: "bob",   password: "secret456",   role: "admin" },
|   { username: "carol", password: "carol1789",   role: "viewer" }
| ])
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': ObjectId('6a343600b85dc5b2bb44ba89'),
|     '1': ObjectId('6a343600b85dc5b2bb44ba8a'),
|     '2': ObjectId('6a343600b85dc5b2bb44ba8b')
|   }
| }
soc_lite> db.users.find({}, { username:1, role:1, _id:0 })
[
| { username: 'alice_analyst' },
| { username: 'bob_senior' },
| { username: 'charlie_admin' },
| { username: 'diana_junior' },
| { username: 'alice', role: 'analyst' },
| { username: 'bob', role: 'admin' },
| { username: 'carol', role: 'viewer' }
| ]
```

FIGURE 1 – Insertion des utilisateurs de test et verification par projection sur les champs `username` et `role`.

Une premiere recherche normale a ensuite ete executee sur l'endpoint `POST /logs/search`. La requete exacte `{"src_ip":"192.168.1.10"}` retourne deux documents. Cette sortie est importante car elle etablit le comportement nominal avant attaque.

```
(Kali@DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/logs/search \
-H "Content-Type: application/json" \
-d '{"src_ip":"192.168.1.10"}' | python3 -m json.tool
{
  "count": 2,
  "data": [
    {
      "_id": "6a29b9c674555d9f1144ba8c",
      "kind": "log",
      "ts": "2026-06-10T08:00:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "login",
      "status": "open",
      "severity": "medium",
      "password_hash": "HASH_FICTIF_SENSIBLE",
      "internal_note": "ticket_17_confidentiel"
    },
    {
      "_id": "6a29b9c674555d9f1144ba90",
      "kind": "log",
      "ts": "2026-06-10T08:30:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "privilege_change",
      "status": "investigating",
      "severity": "critical",
      "internal_note": "role escalation suspected"
    }
  ]
}
```

FIGURE 2 – Recherche normale sur `/logs/search` : deux logs associes a `192.168.1.10` sont retournes.

La sortie montre egalement un probleme volontaire de conception : en l'absence de projection, l'API retourne des champs internes comme `password_hash` et `internal_note`. Cette observation annonce la Partie 4 sur le principe *Least Data*.

Le second comportement nominal teste est l'authentification. Avec le bon couple `alice/motdepasse123`, l'API repond `success:true` et retourne le role `analyst`.

```
(Kali@DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"motdepasse123"}' | python3 -m json.tool
{
  "success": true,
  "username": "alice",
  "role": "analyst"
}
```

FIGURE 3 – Connexion legitime avec identifiants valides : l'API retourne `success:true`.

4.2 Partie 2 — Demonstration controlee des injections

4.2.1 Refus du mauvais mot de passe

Avant de tester l'injection, le mauvais mot de passe a ete verifie. La requete `password:"MAUVAIS_MDP"` retourne `success:false`. Cette etape est utile : elle prouve que l'authentification fonctionne normalement lorsque la valeur envoyee est une chaine simple.

```
(Kali@DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"MAUVAIS_MDP"}' | python3 -m json.tool
{
  "success": false
}
```

FIGURE 4 – Controle negatif : un mauvais mot de passe est refuse par l'API.

4.2.2 Injection `$ne` sur le login

Le test suivant remplace le mot de passe par un objet MongoDB : `{"$ne":""}`. L'API vulnérable construit alors involontairement le filtre suivant :

```
{ username: "alice", password: { $ne: "" } }
```

Ce filtre signifie : chercher l'utilisateur `alice` dont le mot de passe est different de la chaine vide. Comme le mot de passe stocke n'est pas vide, MongoDB retourne le compte et l'API indique une connexion reussie sans connaitre le mot de passe.

```
(Kali@DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":{"$ne":""}}' | python3 -m json.tool
{
  "success": true,
  "username": "alice",
  "role": "analyst"
}
```

FIGURE 5 – Injection `$ne` : l'API vulnérable accepte la connexion malgré l'absence du mot de passe reel.

Impact securite. Dans un contexte SOC, ce type de contournement ne se limite pas a un simple acces non autorise. Les actions suivantes seraient attribuees a l'utilisateur usurpe, ce qui degrade la tracabilite et complique l'investigation forensique.

4.2.3 Injection `$regex` sur la recherche de logs

La deuxieme injection porte sur l'endpoint `/logs/search`. Une recherche exacte par `src_ip` retourne deux documents, tandis que le payload `{"src_ip":{"$regex":""}}` retourne davantage de resultats, car une expression reguliere vide matche toutes les chaines.


```
(KaliⓈ DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/logs/search \
-H "Content-Type: application/json" \
-d '{"src_ip":"192.168.1.10"}' | python3 -m json.tool
{
  "count": 2,
  "data": [
    {
      "_id": "6a29b9c674555d9f1144ba8c",
      "kind": "log",
      "ts": "2026-06-10T08:00:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "login",
      "status": "open",
      "severity": "medium",
      "password_hash": "HASH_FICTIF_SENSIBLE",
      "internal_note": "ticket_17_confidentiel"
    },
    {
      "_id": "6a29b9c674555d9f1144ba90",
      "kind": "log",
      "ts": "2026-06-10T08:30:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "privilege_change",
      "status": "investigating",
      "severity": "critical",
      "internal_note": "role escalation suspected"
    }
  ]
}
```

FIGURE 6 – Injection `$regex` : la regex vide retourne tous les logs dont le champ `src_ip` est une chaîne.

La comparaison avec `mongosh` confirme le phénomène : la requête exacte sur `192.168.1.10` et la requête regex ne produisent pas le même volume.

```
soc_lite> db.security_logs.find({ src_ip: "192.168.1.10" }).count()
0
soc_lite> db.security_logs.find({ src_ip: { $regex: "" } }).count()
11
```

FIGURE 7 – Comparaison des volumes : recherche exacte par IP contre recherche injectée par `$regex`.

4.2.4 Mass assignment et champs non autorisés

Le support traite aussi le mass assignment : appliquer directement `$set:req.body` dans une mise à jour revient à laisser le client choisir les champs modifiés. Les captures montrent un test de mise à jour de profil incluant un champ sensible `role`. Dans une API vulnérable, ce type de payload peut transformer une mise à jour d'email en élévation de privilège.

```

soc_lite> db.security_logs.updateOne(
|   { src_ip: "192.168.1.10" },
|   { $set: { password_hash: "HASH_FICTIF_SENSIBLE", internal_note: "ticket_17_confidentiel" } }
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 0,
|   modifiedCount: 0,
|   upsertedCount: 0
| }

```

FIGURE 8 – Mise a jour incluant un champ `role` : exemple de payload typique de mass assignment.

5 Correction applicative : validation, `safeFilter` et conservation des usages legitimes

5.1 Validation de type

Le premier correctif consiste a rejeter les objets la ou une chaine est attendue. Dans le cas du login, le mot de passe doit etre une `string`. Si le client envoie un objet contenant `$ne`, la validation echoue avant toute requete MongoDB.

```

(KaliⓈ DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":{"$ne":""}}' | python3 -m json.tool
{
  "error": "Invalid credentials format"
}

```

FIGURE 9 – Après correction, le payload `password:{$ne:""}` retourne une erreur de format au lieu d'atteindre MongoDB.

La meme logique s'applique au champ `src_ip`. L'application attend une chaine correspondant a un format IPv4. Un objet `{"$regex":""}` est donc rejete comme entree invalide.

```

(KaliⓈ DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/logs/search \
-H "Content-Type: application/json" \
-d '{"src_ip":{"$regex":""}}' | python3 -m json.tool
{
  "error": "Invalid src_ip"
}

```

FIGURE 10 – Après correction, l'injection `$regex` est rejetee par la validation de `src_ip`.

5.2 Construction explicite du filtre

Le support insiste sur l'interdiction de passer `req.body` directement a `find()` ou `findOne()`. Le filtre doit etre construit champ par champ :

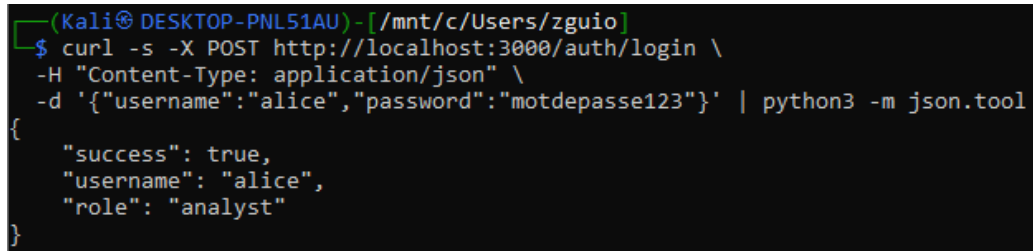
```
const filter = { kind: "log" };
```

```
if (body.src_ip !== undefined) {  
  if (!cleanString(body.src_ip)) throw new Error("Invalid src_ip");  
  filter.src_ip = body.src_ip.trim();  
}
```

Cette approche place l'application en controle complet de la requete. Le client peut proposer une valeur, mais il ne choisit jamais l'operateur MongoDB ni la structure du filtre.

5.3 Verification de non-regression fonctionnelle

Un correctif securite n'est acceptable que s'il bloque l'anormal sans casser le normal. Les captures montrent que le login legitime reste fonctionnel apres correction.



```
(KaliⓈ DESKTOP-PNL51AU)-[ /mnt/c/Users/zguio ]  
$ curl -s -X POST http://localhost:3000/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"alice","password":"motdepasse123"}' | python3 -m json.tool  
{  
  "success": true,  
  "username": "alice",  
  "role": "analyst"  
}
```

FIGURE 11 – Non-regression : le login valide fonctionne toujours apres activation des controles.

La recherche legitime sur `src_ip` continue egalement de retourner les deux logs attendus.

```
(Kali@DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/logs/search \
-H "Content-Type: application/json" \
-d '{"src_ip":"192.168.1.10"}' | python3 -m json.tool
{
  "count": 2,
  "data": [
    {
      "_id": "6a29b9c674555d9f1144ba8c",
      "kind": "log",
      "ts": "2026-06-10T08:00:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "login",
      "status": "open",
      "severity": "medium",
      "password_hash": "HASH_FICTIF_SENSIBLE",
      "internal_note": "ticket_17_confidentiel"
    },
    {
      "_id": "6a29b9c674555d9f1144ba90",
      "kind": "log",
      "ts": "2026-06-10T08:30:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "privilege_change",
      "status": "investigating",
      "severity": "critical",
      "internal_note": "role escalation suspected"
    }
  ]
}
```

FIGURE 12 – Non-regression : la recherche exacte par IP reste disponible apres correction.

6 Projection minimale et principe Least Data

6.1 Observation du probleme sans projection

La projection minimale repond a un risque different de l'injection : la fuite de donnees par sur-exposition. Meme si le filtre est correct, une API qui retourne le document MongoDB complet peut exposer des champs internes inutiles au client.

La capture suivante montre un document lu sans projection. Les champs `password_hash` et `internal_note` apparaissent dans la sortie, alors qu'ils ne devraient pas etre visibles dans une reponse API publique.

```
soc_lite> db.security_logs.findOne({ src_ip: "192.168.1.10" })
{
  _id: ObjectId('6a35a486b85dc5b2bb44ba8c'),
  timestamp: ISODate('2025-03-03T10:12:00.000Z'),
  event_type: 'login_success',
  src_ip: '192.168.1.10',
  dst_ip: '10.0.0.5',
  src_port: 52100,
  dst_port: 443,
  protocol: 'HTTPS',
  user_attempted: 'alice',
  device_id: 'siem-console',
  severity: 'info',
  raw_log: 'Successful login for alice',
  internal_note: 'ticket_17_confidentiel',
  password_hash: 'HASH_FICTIF_SENSIBLE'
}
```

FIGURE 13 – Lecture sans projection : les champs sensibles et internes sont visibles.

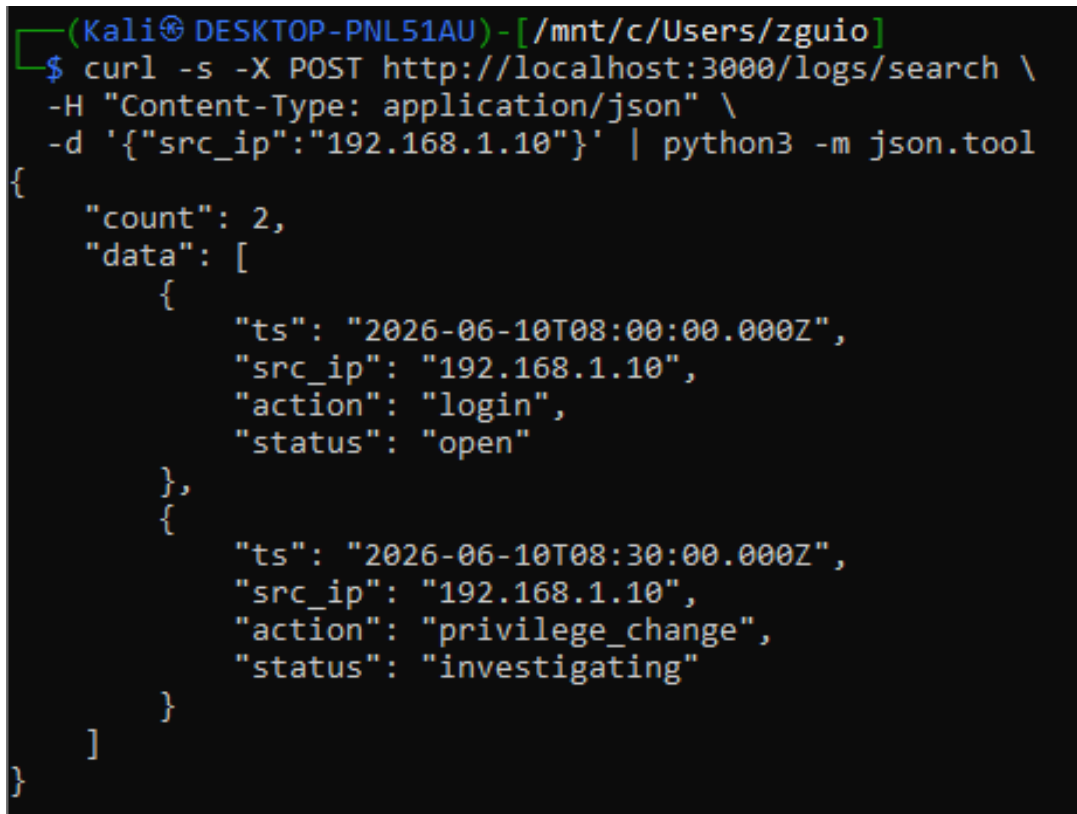
6.2 Lecture avec projection minimale

La correction consiste a declarer explicitement les champs autorises. Pour les logs, l'API n'a besoin que de `ts`, `src_ip`, `action` et `status`. Le champ `_id` est exclu par `_id:0`.

```
soc_lite> db.security_logs.findOne(
|   { src_ip: "192.168.1.10" },
|   { _id:0, ts:1, src_ip:1, action:1, status:1 }
| )
{ src_ip: '192.168.1.10' }
```

FIGURE 14 – Lecture MongoDB avec projection : seuls les champs utiles sont conserves.

L'appel API securise confirme ensuite que la reponse JSON ne contient plus `password_hash`, `internal_note`, `severity`, `kind` ni `_id`.



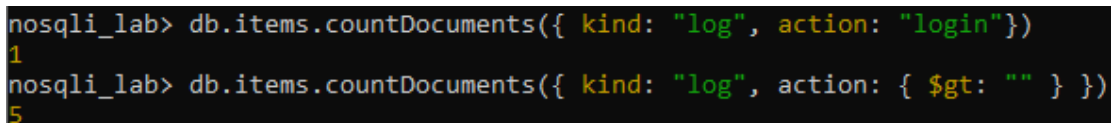
```
(KaliⓈ DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/logs/search \
-H "Content-Type: application/json" \
-d '{"src_ip":"192.168.1.10"}' | python3 -m json.tool
{
  "count": 2,
  "data": [
    {
      "ts": "2026-06-10T08:00:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "login",
      "status": "open"
    },
    {
      "ts": "2026-06-10T08:30:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "privilege_change",
      "status": "investigating"
    }
  ]
}
```

FIGURE 15 – Reponse API avec projection minimale : seulement `ts`, `src_ip`, `action` et `status`.

7 Execution du devoir maison

7.1 Test 1 — Injection `$gt` sur les logs

Le devoir demande de comparer une recherche normale `{"action":"login"}` avec une recherche injectee `{"action":{"$gt":""}}`. Dans l'adaptation legere, les commandes `countDocuments()` montrent que l'action exacte `login` retourne un seul document, alors que `$gt:""` retourne un volume superieur.



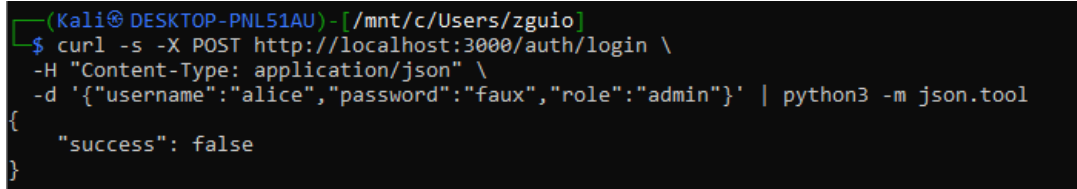
```
nosqli_lab> db.items.countDocuments({ kind: "log", action: "login"})
1
nosqli_lab> db.items.countDocuments({ kind: "log", action: { $gt: "" } })
5
```

FIGURE 16 – Devoir - Test 1 : comparaison des volumes entre `action:"login"` et `action:{$gt:""}`.

La raison est lexicographique : pour des champs de type chaine, beaucoup de valeurs non vides sont superieures a la chaine vide. Le filtre ne signifie donc plus “action egale login”, mais “action non vide ou superieure a vide”, ce qui elargit considerablement la recherche.

7.2 Test 2 — Champ non prevu dans le login

Le test suivant envoie `role:"admin"` avec un mauvais mot de passe. Dans l'implementation observee, ce champ n'affecte pas le resultat : l'endpoint extrait explicitement `username` et `password`. La reponse reste `success:false`.



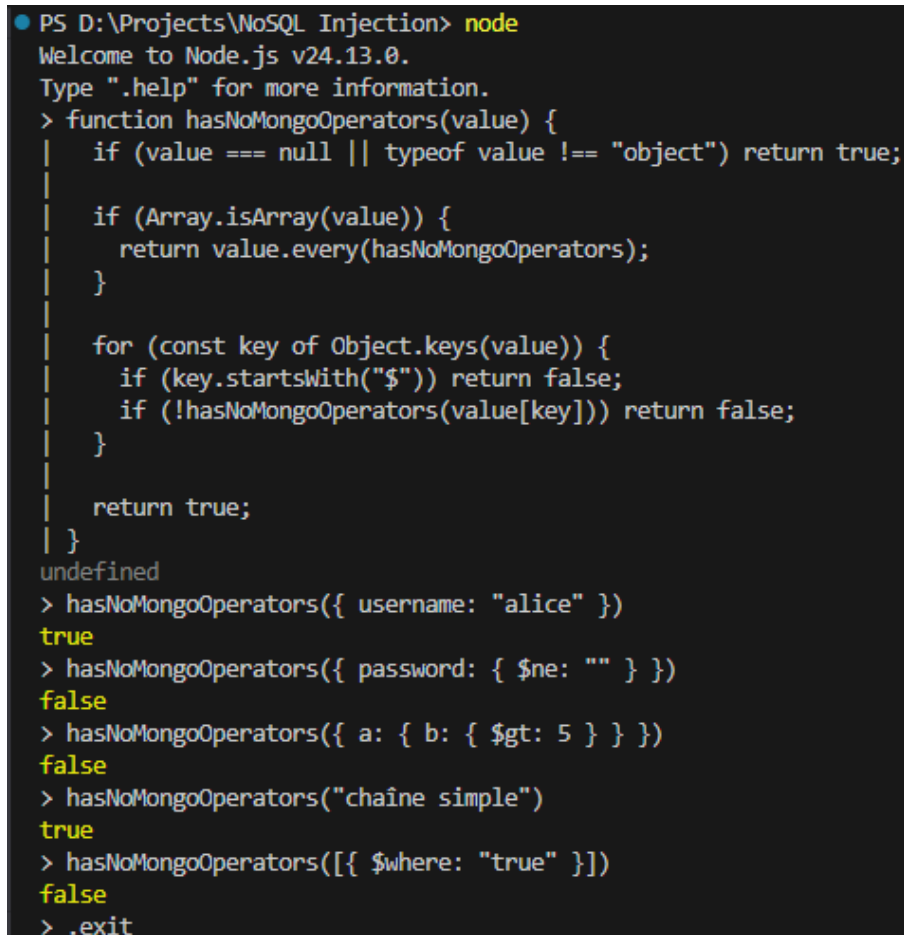
```
(Kali@DESKTOP-PNL51AU)-[ /mnt/c/Users/zguio ]
$ curl -s -X POST http://localhost:3000/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"alice","password":"faux","role":"admin"}' | python3 -m json.tool
{
  "success": false
}
```

FIGURE 17 – Devoir - Test 2 : le champ `role` n'influence pas l'authentification dans cet endpoint.

Ce resultat ne signifie pas que `role` est sans danger. Il montre seulement que cet endpoint precis n'utilise pas `req.body` comme filtre complet. Le meme champ deviendrait critique dans une route de mise a jour vulnerable utilisant `$set:req.body`.

7.3 Test 3 — Fonction `hasNoMongoOperators()`

La fonction `hasNoMongoOperators()` a ete testee en Node REPL. Les resultats attendus sont observes : un objet simple retourne `true`, tandis que des objets contenant `$ne` ou `$gt` retournent `false`. La version testee inspecte aussi les tableaux recursivement ; le cas `[{$where:"true"}]` retourne donc `false`, ce qui est le comportement souhaitable.



```
PS D:\Projects\NoSQL Injection> node
Welcome to Node.js v24.13.0.
Type ".help" for more information.
> function hasNoMongoOperators(value) {
|   if (value === null || typeof value !== "object") return true;
|
|   if (Array.isArray(value)) {
|     return value.every(hasNoMongoOperators);
|   }
|
|   for (const key of Object.keys(value)) {
|     if (key.startsWith("$")) return false;
|     if (!hasNoMongoOperators(value[key])) return false;
|   }
|
|   return true;
| }
undefined
> hasNoMongoOperators({ username: "alice" })
true
> hasNoMongoOperators({ password: { $ne: "" } })
false
> hasNoMongoOperators({ a: { b: { $gt: 5 } } })
false
> hasNoMongoOperators("chaîne simple")
true
> hasNoMongoOperators([{$where: "true"}])
false
> .exit
```

FIGURE 18 – Devoir - Test 3 : verification recursive de l'absence d'opérateurs MongoDB.

7.4 Test 4 — Effet de la projection avec et sans `project()`

Le devoir demande de commenter temporairement la projection, de lister les champs supplementaires visibles, puis de remettre la projection. Les captures du travail montrent les deux situations : sans

projection, le document contient les champs internes ; avec projection, ils disparaissent.

```
(Kali@ DESKTOP-PNL51AU) - [ /mnt/c/Users/zguio ]
$ curl -s -X POST http://localhost:3000/logs/search \
-H "Content-Type: application/json" \
-d '{"action":{"$gt":""}}' | python3 -m json.tool
{
  "count": 5,
  "data": [
    {
      "_id": "6a29b9c674555d9f1144ba8c",
      "kind": "log",
      "ts": "2026-06-10T08:00:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "login",
      "status": "open",
      "severity": "medium",
      "password_hash": "HASH_FICTIF_SENSIBLE",
      "internal_note": "ticket_17_confidentiel"
    },
    {
      "_id": "6a29b9c674555d9f1144ba8d",
      "kind": "log",
      "ts": "2026-06-10T08:04:00.000Z",
      "src_ip": "192.168.1.15",
      "action": "scan",
      "status": "investigating",
      "severity": "high",
      "password_hash": "HASH_FICTIF_SENSIBLE_2",
      "internal_note": "possible reconnaissance"
    },
    {
      "_id": "6a29b9c674555d9f1144ba8e",
      "kind": "log",
      "ts": "2026-06-10T08:10:00.000Z",
      "src_ip": "10.0.0.8",
      "action": "failed_login",
      "status": "open",
      "severity": "low",
      "internal_note": "3 failed attempts"
    },
    {
      "_id": "6a29b9c674555d9f1144ba8f",
      "kind": "log",
      "ts": "2026-06-10T08:20:00.000Z",
      "src_ip": "172.16.2.20",
      "action": "malware_alert",
      "status": "closed",
      "severity": "critical",
      "internal_note": "quarantined"
    },
    {
      "_id": "6a29b9c674555d9f1144ba90",
      "kind": "log",
      "ts": "2026-06-10T08:30:00.000Z",
      "src_ip": "192.168.1.10",
      "action": "privilege_change",
      "status": "investigating",
      "severity": "critical",
      "internal_note": "role escalation suspected"
    }
  ]
}
```

FIGURE 19 – Devoir - Test 4 : sortie longue sans projection, montrant l'exposition de champs supplémentaires.

Les champs à ne pas exposer dans une API publique sont notamment : `_id`, `password_hash`, `internal_note`, tout secret applicatif, tout identifiant interne inutile et tout champ d'audit non destiné au client.

7.5 Test 5 — Endpoint `/users/update-email`

Le dernier test consiste à ajouter une route de mise à jour d'email avec validation de type, validation de format, allow-list des champs modifiables et projection minimale en sortie. La première capture montre une mise à jour légitime de l'adresse de `carol`.


```
(Kali@DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/users/update-email -H "Content-Type: application/json"
-d '{"username":"carol","email":"carol.new@soc.local"}' | python3 -m json.tool
{
  "success": true,
  "user": {
    "username": "carol",
    "role": "viewer",
    "email": "carol.new@soc.local"
  }
}
```

FIGURE 20 – Devoir - Test 5 : mise a jour legitime de l'email de carol.

La seconde capture montre une tentative de mass assignment avec `role:"admin"`. La reponse confirme que l'email est traite, mais le role ne fait pas partie de l'objet `$set` autorise. La route applique donc correctement l'allow-list.

```
(Kali@DESKTOP-PNL51AU)-[/mnt/c/Users/zguio]
$ curl -s -X POST http://localhost:3000/users/update-email -H "Content-Type: application/json"
-d '{"username":"carol","email":"nouveau@test.com","role":"admin"}' | python3 -m json.tool
{
  "success": true,
  "user": {
    "username": "carol",
    "role": "viewer",
    "email": "nouveau@test.com"
  }
}
```

FIGURE 21 – Devoir - Test 5 : tentative de modification du role ignoree par l'allow-list.

8 Checklist securite remplie a partir des preuves

Controle	Preuve observee	Etat
Validation de type	Les payloads <code>password:{\$ne:""}</code> et <code>src_ip:{\$regex:""}</code> retournent des erreurs applicatives.	Valide
Construction explicite des filtres	Les requetes legitimes continuent de fonctionner apres correction, sans passer l'objet utilisateur complet a MongoDB.	Valide
Blocage des operateurs \$	La fonction <code>hasNoMongoOperators()</code> detecte <code>\$ne</code> , <code>\$gt</code> et <code>\$where</code> .	Valide
Projection minimale	La reponse finale ne contient que <code>ts</code> , <code>src_ip</code> , <code>action</code> , <code>status</code> .	Valide
Protection contre mass assignment	L'endpoint <code>/users/update-email</code> ignore le champ <code>role</code> .	Valide
Non-regression fonctionnelle	Le login valide et la recherche exacte par IP restent fonctionnels.	Valide
Gestion des erreurs	Les erreurs sont transformees en messages JSON applicatifs ; point a renforcer en production pour eviter <code>err.message</code> brut.	A renforcer
Stockage des mots de passe	Les mots de passe de test sont en clair dans le TP ; en production, il faudrait <code>bcrypt</code> ou <code>argon2</code> .	Pedagogique

9 Analyse critique et enseignements

9.1 1. Le probleme est applicatif avant d’etre base de donnees

Le TP montre clairement que MongoDB execute le filtre qu’il recoit. La base ne peut pas savoir si `$ne` ou `$regex` vient d’un administrateur legitime ou d’un attaquant. La frontiere de securite se trouve donc dans l’API : validation des entrees, construction du filtre et limitation des champs retournees.

9.2 2. L’injection NoSQL n’est pas une injection SQL traduite

L’enseignement central est la nature objet de l’attaque. En SQL, l’attaquant cherche souvent a sortir d’une chaine et a injecter une condition. En MongoDB, il peut envoyer un objet JSON valide contenant des operateurs. C’est pourquoi les “prepared statements” SQL ne se transposent pas directement. L’equivalent pratique est la construction explicite d’un filtre a partir de valeurs validees.

9.3 3. La projection est une defense independante

Bloquer l’injection ne suffit pas si la reponse retourne trop d’informations. Les captures avec et sans projection montrent un risque concret : des champs sensibles peuvent sortir par une API pourtant fonctionnellement correcte. La projection minimale doit donc etre consideree comme une couche de defense separee.

9.4 4. Mongoose ou un ORM/ODM ne remplacent pas la validation

Un ODM comme Mongoose peut aider par schema, casting, validation et selection de champs, mais il ne supprime pas le risque si le developpeur ecrit encore `Model.find(req.body)` ou `$set:req.body`. La discipline de construction explicite du filtre reste necessaire.

10 Conclusion

La Seance 4 a permis de couvrir tout le cycle d’une vulnerabilite NoSQL Injection : identification du point faible, reproduction controlee, mesure de l’impact, correction applicative et verification de non-regression. Les preuves montrent que les attaques `$ne`, `$regex` et `$gt` sont bien comprises, et que les mecanismes de defense ont ete appliques de maniere coherente.

Le livrable final met surtout en evidence un principe de fond : la securite MongoDB ne se limite pas a la configuration du serveur. Elle depend aussi de la facon dont l’API construit les requetes, gere les mises a jour, filtre les operateurs et minimise les donnees retournees. Pour une application de production, cette couche doit etre completee par le hashing des mots de passe, la gestion robuste des secrets, l’authentification forte, le RBAC, la journalisation, le rate limiting et une politique d’erreurs non verbeuse.

A Extraits representatifs des commandes executees

A.1 Requetes normales

```
curl -s -X POST http://localhost:3000/logs/search \
```

```
-H "Content-Type: application/json" \  
-d '{"src_ip":"192.168.1.10"}' | python3 -m json.tool  
  
curl -s -X POST http://localhost:3000/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"alice","password":"motdepasse123"}' | python3 -m json.tool
```

A.2 Payloads d'injection

```
curl -s -X POST http://localhost:3000/auth/login \  
-H "Content-Type: application/json" \  
-d '{"username":"alice","password":{"$ne":""}}' | python3 -m json.tool  
  
curl -s -X POST http://localhost:3000/logs/search \  
-H "Content-Type: application/json" \  
-d '{"src_ip":{"$regex":""}}' | python3 -m json.tool  
  
curl -s -X POST http://localhost:3000/logs/search \  
-H "Content-Type: application/json" \  
-d '{"action":{"$gt":""}}' | python3 -m json.tool
```

A.3 Projection minimale

```
db.security_logs.findOne(  
  { src_ip: "192.168.1.10" },  
  { _id: 0, ts: 1, src_ip: 1, action: 1, status: 1 }  
)
```

A.4 Endpoint update-email securise

```
curl -s -X POST http://localhost:3000/users/update-email \  
-H "Content-Type: application/json" \  
-d '{"username":"carol","email":"carol.new@soc.local"}' | python3 -m json.tool  
  
curl -s -X POST http://localhost:3000/users/update-email \  
-H "Content-Type: application/json" \  
-d '{"username":"carol","email":"nouveau@test.com","role":"admin"}' | python3 -m  
↪ json.tool
```